LLM-based Compatibility Detection

***Abstract*—xxxxxxx[?]**

I. Introduction

II. Background

A. API Compatibility Detection

The research necessity of API compatibility detection is driven by the combination of practical pain points of developers, technical evolution challenges, industry value demands, and gaps in existing solutions. In open-source communities such as GitHub, a large number of developer complaints indicate that version changes of third-party dependency package APIs often cause project errors—for example, runtime errors resulting from interface parameter adjustments or method deprecation. These issues not only increase debugging costs but also may disrupt project iteration. The high-frequency occurrence of such problems in open-source projects directly reflects the urgent need for efficient detection tools. With the popularization of microservices and multi-cloud architectures, systems have become increasingly dependent on third-party APIs; a single project may involve dozens or even hundreds of API versions. Mixed scenarios involving cross-protocols (e.g., REST, gRPC) and multi-data formats (e.g., JSON, PROTOBUF) further exacerbate the difficulty of version management and compatibility verification. Traditional manual detection or mainstream tools (such as Postman and Swagger) focus on functional testing, making it difficult to cover implicit compatibility issues (e.g., semantic-level differences, residual implicit dependencies) and unable to meet detection needs in complex architectures. From the perspective of industry value, API compatibility has become a core element in ensuring system stability—in critical fields such as finance and healthcare, compatibility issues may lead to severe consequences such as transaction interruptions and data errors, resulting in direct economic losses. In cross-enterprise collaboration, version incompatibility prolongs the system integration cycle and reduces the efficiency of ecological collaboration. However, existing technologies lack sufficient capability to analyze API semantic differences and struggle to address detection needs in scenarios involving dynamically generated APIs and multi-role usage. This highlights the research value of detection solutions based on LLMs (Large Language Models) combined with COT (Chain of Thought) and MultiAgent technologies: through logical reasoning to analyze the semantic impact of changes and multi-perspective simulation for collaborative verification, these solutions are expected to break through the limitations of traditional schemes, solve the practical pain points of developers and technical bottlenecks in complex architectures, and hold significant theoretical and practical significance for ensuring system reliability and improving industry collaboration efficiency.

B. API Compatibility Type

C. Motivating Example

III. Approach

This section details our methodology for detecting Python API incompatibilities using Large Language Models (LLMs) with structured Chain-of-Thought (COT) prompting. We first outline the overall workflow and core challenges, then elaborate on the design principles compared to existing methods, and technical implementation This section details our methodology for detecting Python API incompatibilities using Large Language Models (LLMs) with structured Chain-of-Thought (COT) prompting. We first outline the overall workflow and core challenges, then elaborate on the design principles compared to existing methods, and technical implementation

A. CoT

B. Overview and Challenges

Our approach targets code-level changed API pairs (excluding module-level, class-level, and API addition/deletion changes) between two versions of a Python project. Figure x illustrates the end-to-end workflow. First, we extract all APIs from two versions of the project and filter API pairs to retain only those with code-level modifications. Then we instruct LLM with COT guidance to performs line-by-line code analysis. Finally, The LLM generates a structured JSON output, including detected incompatibility types, detailed reasons for each type, and auxiliary information.

figure x

During this three-step process, we need to address the following three challenges:

- **High-Level Description Misalignment (C1)**: Existing LLM-based methods[x] often rely on high-level incompatibility descriptions rather than code-level semantics. This disconnect causes LLMs to struggle with mapping abstract descriptions to concrete code changes.
- **Scalability with Incompatibility Categories (C2)**: As the number of incompatibility categories increases, naive prompting leads to exponential growth in prompt token size and LLM runtime. For N categories, this approach requires N independent LLM calls per API pair, making it infeasible for large-scale analysis.
- **Unreliable and Inconsistent Outputs (C3)**: LLMs exhibit inherent output randomness—they may produce contradictory judgments for the same prompt (e.g., labeling a parameter rename as "compatible" in one run and "incompatible" in another) or include internally inconsistent reasoning (e.g., citing non-existent code lines to support a claim).
- **Unstructured COT Reasoning (C4)**: Without guided thinking frameworks, LLMs' intermediate reasoning (e.g.,

how they identify a parameter default value change) remains unstructured. This makes it impossible to verify the validity of the LLM's conclusions.

C. Methodology Design

To address the above challenges, we design a one-pass COT analysis framework with structured prompting, built on insights from prior empirical studies and innovations in LLM guidance.

We first establish a comprehensive taxonomy of Python API incompatibilities, extending the 7-category framework proposed by Shen et al. [x] (a foundational empirical study on Python API evolution). Shen et al. identified incompatibilities rooted in parameters (CI1-CI5), exceptions (CI6), and return values (CI7). Through our own data collection, we validated these 7 categories and discovered two new, previously unreported types:

CI8: global variable change **CI9**: reference variable change Table x

1) Limitation of Current Methods: Traditional static analysis methods (e.g., Shen et al. [2024]) rely on handcrafted rules. However, Rule design requires expert knowledge making it hard to generalize to new libraries or unknown incompatibilities. For example, shen define data structure P to record parameter information for the detection of CI1-CI5. However, without detailed study into incompatible examples. it is hard to design rules for detections. Besides, These methods fail to interpret semantic context while many crucial infomation should be analyzed through variable names, comments, usage in function bodies, etc. For example, Shen et al.'s method cannot recognize that renaming a parameter from arguments to args is a compatibility issue—since the AST pattern for parameter names differs, but the static rule does not link the semantic similarity between arguments and args.

In our method, by leveraging LLMs' semantic understanding capabilities, our approach directly addresses this limitation. The LLM integrates contextual cues to identify semantic changes, even when syntactic patterns are non-trivial.

2) One-Pass COT Analysis Framework: To tackle scalability and output unreliability, we design a one-pass COT framework guided by two core principles: two-stage analysis and unified scanning. First, the LLMs is instructed to process the parameter list, which is syntactically structured, then analyze the function body, which is semantically rich—aligning with how developers naturally review code changes. We embed rules for all 9 incompatibility categories into a single prompt to enable the LLM to detect multiple incompatibilities in one analysis pass, avoiding repeated LLM calls. The framework is divided into four sequential phases, each with predefined data structures and rules to guide the LLM's reasoning. This design offers three key advantages: efficiency, as runtime is independent of the number of incompatibility categories (all rules are processed in one pass); reduced misjudgment, as it enforces dependencies between interdependent categories (e.g., analyzing CI1 (parameter addition) before CI4 (default value change) to avoid misclassifying added parameters as default value changes); and structured reasoning, as each phase outputs intermediate data structures like parameter mappings and exception type lists, which make the LLM's logic traceable and verifiable.

3) Structured Prompt Design: The prompt is the backbone of our framework, structured to enforce consistency and guide the LLM's COT reasoning. It includes four components: Instructions, Analysis Phases, Source Code, and Examples. A simplified template is shown below (full rules are provided in Appendix A):

```
Analyze two versions of a Python API
    function (‘code_v1‘ and ‘code_v2‘) to
    detect incompatibilities. The output
    must be in JSON format.

    ### Instructions
    Please strictly follow the ‘Analysis
        Phases‘ to identify all
        incompatibility types (CI1 – CI9)
        and provide concise reasons. The
        final output must include:
    – ‘additional_info‘: Key variables/
        structures used in analysis (e.g.,
         parameter mappings).
    – ‘reasons‘: One–sentence
        explanations linking code changes
        to incompatibility types.
    – ‘incompatibility‘: List of detected
         types

    ### Analysis Phases
    #### Phase 0: Initialization
    Initialize the result dictionary:
    {
    ”incompatibility”: [], ”reasons”: [],
         ”additional_info”: {”
        param_mapping”: {}, ”
        exception_list_v1”: [], ...}
    }
    #### Phase 1: Parameter List
        Processing
    Extract parameter lists from ‘code_v1
        ‘ and ‘code_v2‘, then build:
    #### Phase 2: Initial Incompatibility
         Analysis
    Use ‘P1‘/‘P2‘ to update results:
    #### Phase 3: Function Body
        Processing (Line–by–Line)
    Initialize data structures for ‘
        code_v1‘/‘code_v2‘:
    #### Phase 4: Comprehensive
        Incompatibility Analysis
    Use Phase 1–3 data structures to
        detect remaining types:

    ### Source Code
    ”code_v1”: ...
    ”code_v2”: ...
```

```
### Examples
}
```

4) Data Structure and Detection Design: The analysis leverages several core data structures to capture critical information from function signatures and bodies, facilitating subsequent incompatibility checks. Based on the detecing process for different incompatibility types, we design following data structure which is initialized at begining and update throughout the scanning of the source code.

Parameter Lists (P1, P2): These lists encode the parameters of source code. Each parameter is represented as a tuple (name, position index, type, default value), where type includes positional, optional (with a default value), or kwargs, enabling direct comparison of parameter presence, order, and defaults.

KWARGS: A string storing the name of the kwargs parameter if present in both code v1 and code v2; empty otherwise. This structure anchors the analysis of keyword argument keys accessed within the function body.

ReName: A list of tuples (name1, name2): name1 (from code v1) and name2 (from code v2) are semantically similar or share the same position index, indicating potential parameter renaming.

Keyword Argument Keys (K1, K2): Lists of keys explicitly accessed via kwargs (e.g., kwargs.get('key') or kwargs['key']) in code v1 and code v2, respectively. These capture dependencies on dynamic keyword arguments.

Renamed Parameter Usage (ReNUsage1, ReNUsage2): Dictionaries mapping parameters in ReName pairs to lists of code lines where they are used. This tracks contextual usage to validate renaming.

Condition Statements (C1, C2): Lists of condition strings (e.g., if x ¿ 0) extracted from code v1 and code v2, indexed sequentially to link with dependent logic (e.g., assignments, exceptions).

Data Flow (D1, D2): Lists of tuples (variable name, variable type, dependent variables, condition indices) capturing variable assignments. variable type distinguishes local variable, global variable, and reference variable (e.g., self), while dependent variables and condition indices track dependencies and associated conditions.

Exceptions (E1, E2): Lists of tuples pairing explicit exception types (e.g., ValueError) with indices of conditions under which they are raised (from C1/C2), excluding implicit behaviors or unthrown exceptions.

Return Values (R1, R2): Lists of tuples pairing return expressions with condition indices, capturing how return values vary under different conditions.

The 9 types of incompatibility can be detected based on the data structure above. The detection rules are showed in Table x.

Table x

D. Multi-Agent-Based Incompatibility Category Validation

To address judgment biases in the incompatibility category candidates generated by the CoT-LLM, we design a multi-agent collaborative validation framework. It implements accurate verification of candidate categories through a process of "test case generation, simulation execution, and comprehensive evaluation," ultimately filtering out truly valid API incompatibility types. The framework comprises three functionally independent yet collaboratively working agent modules: the Test Case Generation Agent, Simulation Agent, and Comprehensive Analysis Agent. These modules form a validation chain through explicit information interaction, ensuring the objectivity and reliability of validation results.

The Test Case Generation Agent serves as the initiator of the validation process, tasked with constructing targeted test inputs to probe potential incompatibilities. Its core functionality is to generate pairs of calling statements—one for the old API version and one for the new—along with supplementary context, based on the provided old and new API code snippets, the candidate incompatibility category, and the underlying reasoning. The design of these test cases adheres to a critical principle: they must be highly likely to trigger the alleged incompatibility if it indeed exists. For instance, when evaluating a candidate issue related to changed default parameter values, the agent will craft calling statements that omit the affected parameters, thereby exposing discrepancies in behavior caused by the default value shift. The output of this agent follows a structured format, explicitly separating the calling statements and supplementary information for the old and new APIs to ensure clarity in subsequent processing.

Upon receiving the test cases from the Test Case Generation Agent, the Simulation Agent takes on the role of executing these tests in a controlled environment. Its primary responsibility is to simulate the runtime behavior of both the old and new API versions when invoked with the generated test inputs. This involves parsing the API code, instantiating necessary objects (if applicable), and executing the calling statements step-by-step. The agent meticulously records the execution results: if the program exits normally with a return value, it captures the specific output; if an exception is raised (e.g., missing arguments, type errors), it documents the error type and description. By faithfully replicating runtime scenarios, this agent transforms abstract code differences into concrete behavioral outcomes, providing empirical data for validating the alleged incompatibility.

The Comprehensive Analysis Agent acts as the final evaluator, synthesizing information from the preceding stages to assess the validity of the candidate incompatibility category. It takes as input the definition of the incompatibility type, the reasoning behind the initial diagnosis, and the behavioral results generated by the Simulation Agent. Using a scoring mechanism ranging from 0 to 10, the agent evaluates the alignment between the diagnosed incompatibility, the test case design, and the observed runtime behaviors. The score is determined based on three key criteria: logical consistency (whether the reasoning logically leads to the alleged incompatibility), rigor of evidence (whether the simulation results clearly demonstrate the expected behavioral discrepancy), and coherence (whether all components of the validation process reinforce the same conclusion). A higher score indicates greater confidence in the existence of the specific incompatibility, while a lower score suggests either flaws in the initial diagnosis or inadequacies in the test scenario.

The collaborative workflow of the three agents forms a closed-loop validation pipeline. Starting with a candidate incompatibility category, the Test Case Generation Agent tailors probes to expose potential issues, the Simulation Agent generates empirical behavioral data, and the Comprehensive Analysis Agent renders a data-driven judgment. This division of labor ensures that each stage of validation is handled by a specialized component, reducing the risk of subjective biases that might arise from a single model's judgment. By structuring the validation as an explicit information flow—from test design to execution to analysis—the framework enhances the transparency and reproducibility of the verification process. Ultimately, this multi-agent approach strengthens the reliability of incompatibility category identification, providing a robust mechanism to filter out spurious candidates and retain only those that are empirically validated.

E. implementation

We implement the approach using Python, leveraging open-source tools and APIs to balance reproducibility and efficiency. For API extraction, we combine the GitHub API and Python's standard ast library: the GitHub API fetches source code for specific project versions to enable automated data collection, while the ast library builds ASTs for function-level slicing—extracting only the function body (excluding imports or global statements outside the API). For LLM interaction, we use DeepSeek-chat API with temperature 0 to minimize randomness and max-tokens 2048 to accommodate structured outputs.

IV. Evaluation

A. Dataset Construction

The Python third-party library Application Programming Interface (API) compatibility issue dataset constructed in this study comprises two core components: a benchmark dataset derived from existing research, which includes 108 incompatible API pairs built by Shen et al. on the Flask and pandas libraries; and an extended dataset supplementary constructed in this work. Through systematic repository screening, Issue/Pull Request (PR) analysis, and multi-round validation, the extended dataset adds over 70 valid samples, resulting in a total of more than 130 compatibility issue annotations. The dataset comprehensively covers third-party libraries across multiple application domains, such as web development, data analysis, and cloud services, and includes API compatibility issues across major, minor, and patch version transitions. This design provides multi-scenario and fine-grained experimental data support for research on Python third-party library API compatibility detection.

Dataset A.

To construct the dataset of 108 incompatible API pairs, Shen et al. selected six version pairs from two representative Python third-party libraries (Flask for web development and pandas for data analysis), covering cross-major, cross-minor, and cross-patch version transitions. A combined approach of regression testing and version update log analysis was adopted to ensure the comprehensiveness and reliability of the dataset. For regression testing, test cases from the previous

TABLE I: Compatibility Incompatibility Categories (CI) — detection & rules

CI	Name	Data-structure construction rules	Incompatibility ide rules
CI1	Parameter Addition / Deletion	Signature-level: parameter appears or disappears between versions. Build P1 and P2: list parameters as tuples (name, position index, type, default).	Trigger when: a par ists in P1 but not i positional paramete P2 but not in P1; or optional param in P2 and causes positiona
CI2	Kwargs Key Addition / Deletion	If both versions have '**kwargs', set 'KWARGS' = param name; extract K1 and K2: explicit keys accessed via 'kwargs.get' or 'kwargs['key']'. Statements to detect include body expressions like 'kwargs.get('key')' or 'kwargs['key']'.	Trigger when 'KW non-empty and K1 ≠ added or removed).
CI3	Parameter Rename	Produce ReName pairs (name1, name2) by semantic or positional match; collect ReNUsage1/2: lines where each renamed parameter is used; collect D1/D2 for data-flow context. Detect from signature name pairs and body lines that use the parameter (assignments/usages).	Trigger when a ReNa ists and ReNUsage p highly similar (iden or structurally iden flow where only vari differ).
CI4	Default Value Change	Compare P1 and P2 default value fields for same-named parameters (or matched by position if appropriate). Statements to detect: function signature default values.	Trigger when a paran in both P1 and P2 bu default values differ
CI5	Parameter Range / Condition Change (affects exceptions)	Build C1/C2 (condition strings), E1/E2 (explicit raises with condition index), and D1/D2 to map conditions to parameters. Statements to detect include condition statements ('if', 'while', 'try' scopes) and 'raise' / 'assert' lines referencing parameters.	Trigger when except conditions tied to a present in both versi so that raising behavi checks differ.
CI6	Exception Add/Delete or Type Change	Record E1/E2: explicit exception types with associated condition indices. Statements to detect: 'raise ExceptionType(...)' and 'assert' lines in function body.	Trigger when except differ between E1 an added, removed, or If len(E1)≠len(E2), (and often CI5 as w
CI7	Return Value Change	Collect R1/R2: return expressions with condition indices; use D1/D2 to trace data-flow dependencies of returned variables. Statements to detect: 'return' (or 'yield') statements and their returned expressions.	Trigger when a retu sion in one version in the other or whe flow producing the r differs in a way tha content/type/structur
CI8	Global Variable Change	In D1/D2, mark entries with variable type = 'global variable' and record dependent variables and conditions. Statements to detect: assignment statements referencing names classified as global; 'global' / 'nonlocal' markers.	Trigger when glob data-flow differs be sions and those diffe change the global va ment or visible state
CI9	Reference Parameter Change (e.g., 'self')	In D1/D2, mark entries with variable type = 'reference variable' (including 'self'); collect attribute assignment sites and dependency lists. Statements to detect: assignment statements to reference variables or their attributes, direct reassignments of reference param.	Trigger when: the re rameter itself is rea new attribute is assig version but not the an existing attribute has materially differ ment logic/dependen

version (v1) of each library were executed on the updated version (v2); failed test cases indicated potential incompatible APIs, leading to the identification of 70 incompatible API pairs from 58 failed test files among 2,290 regression test files across the six version pairs. Complementarily, version update logs (including intermediate logs for cross-patch version pairs to capture persistent changes) were analyzed to identify additional incompatible APIs by targeting keywords such as "parameter reduction", "added processing", and "no longer supported", resulting in 38 supplementary incompatible API pairs from 11 analyzed logs. To enhance data credibility, the collection process was independently conducted by the first two authors, with discrepancies resolved through group discussions organized by the third author, involving joint reviews of source code, regression test results, and update logs to confirm the existence of compatibility issues for controversial API pairs. This dual-method strategy effectively mitigated limitations associated with incomplete test coverage in unit testing and inconsistent log quality, ultimately forming a dataset of 108 incompatible API pairs that served as the foundation for subsequent empirical analysis.

Dataset B.

We adopted a different strategy to construct our own dataset to cover codes from more areas. We first selected the top 10 Python libraries by download volume over the past 30 days (as of May 1, 2025) from "https://hugovk.github.io/top-pypi-packages/top-pypi-packages-30-days.json". Those libraries are the most popular ones and can represent most python code usage. However, to cover most compatibility issues, for each candidate repository, we also selected additional relevant Python repositories because some issues are not directly reported in the source repository but in relevant Python repositories.

In the preliminary screening of Issues and PRs, for each selected repository, we cycled through a list of keywords (including 'compatibility', 'compatible', 'version', 'evolution', 'exception', 'TypeError', 'AttributeError', 'ImportError', 'breaking changes') and used the GitHub Issues Search API (/search/issues) to retrieve issues and PRs matching these keywords.

Thereafter, approximately 400 entries were filtered through two rounds of LLM-based checks, followed by manual review. In the LLM analysis phase, the LLM was employed to extract structured information from the issues, including username, repository, version tag1, version tag2, file path, object name, and llm analyse compatibility. Multiple GitHub checks were conducted on the extracted results to ensure the validity of the fields. Specifically, the first LLM was used to convert raw issues into structured compatibility problem entries, which constituted the core information extraction of the dataset, extracting key information such as error type, root cause, library, version, affected files, breaking change, and recommended version. Additionally, the second LLMs was used to supplement and explain the technical details of compatibility issues, especially the causal chain between API changes and errors, serving as an in-depth evidence supplement of the dataset.

After filtering of LLMs, from the original 2000 samples, xxx was ...

Finally, manual verification was performed to confirm the existence of incompatibilities and the correctness of Python API paths, followed by annotation. Ultimately, over 70 samples were obtained, with a total of more than 130 annotations added.

TABLE II: Top Python Libraries by Download Volume

oprule Library	Category	relevant project counts
boto3	Web development	9
urllib3	Web development	7
requests	Web development	10
botocore	Web development	6
certifi	Security	3
charset	xxx	0
normalizer	xxx	0
idna	xxx	3
setuptools	xxx	10
aiobotocore	xxx	3
python-dateutil	xxx	2

B. Experiment Setup

- **Accuracy Evaluation (RQ1)**: How is the overall accuracy of TOOL compared with the state-of-the-arts?
- **Effectiveness Evaluation (RQ2)**: How is the effectiveness of TOOL in different API compatibility types?
- **LLM Generalizability (RQ3)**: How is the accuracy of TOOL using different LLMs?
- **Ablation Study (RQ4)**:

Baselines. We selected XXX , XXX, and XXX as the comparison.

Metrics. We use precision and recall as metrics, which are defined as follows:

C. Results

1) Accuracy Evaluation (RQ1):

2) RQ2:

3) RQ3:

4) RQ4:

V. Related Work

A. Traditional Compatibility Detection Methods

API compatibility of Python third-party libraries is a critical concern given their widespread use in diverse domains, and relevant detection methods have been a focus of research. Lamothe et al. (2021)[**?**] conducted a systematic review of 369 API evolution publications (1994–2020), noting early work's focus on Java (63.9%) and gaps in dynamic language support, uniform benchmarks, and fine-grained incompatibility analysis—findings that guided Python-specific method development.

Early work in this space focused on static analysis to model API lifecycles, but with limited granularity. Quan et al. (2022)[**?**] pioneered this direction for Python with PyMevol, a static tool that constructs a history-sensitive, alias-aware API explorer tree. PyMevol effectively tracked API introductions, modifications, and removals, and addressed Python's unique transitive import-induced alias problem—an issue overlooked by earlier static tools for statically typed languages. However, PyMevol's focus on high-level API lifecycle tracking left a gap in identifying fine-grained causes of incompatibilities (e.g.,

parameter range changes, exception type shifts). This limitation was addressed by Shen et al.[**?**], who proposed a more precise static analysis method: by leveraging statement-level dataflow and control-flow slicing, their work extracted detailed incompatibility causes (e.g., parameter additions/deletions, default value modifications, return type changes) that PyMevol could not capture, thereby enhancing the actionable insights for developers.

While static analysis methods (Quan et al., Shen et al.) excelled at single-project syntactic detection, they struggled to uncover semantic-level incompatibilities across multiple client projects—a critical scenario for real-world library usage. Chen et al. (2020)[**?**] addressed this gap with DeBBI, which transformed behavioral backward incompatibility (BBI) detection into an information retrieval problem. Unlike PyMevol and Shen et al.'s single-project focus, DeBBI prioritized client projects with high relevance to API changes (via TF-IDF and LDA models) and optimized test execution via regression test selection. This approach not only resolved the inefficiency of large-scale cross-project testing but also detected semantic BBIs (e.g., undocumented behavioral shifts) that static syntactic analysis missed.

There are also some studies focusing on conflicts between repositories (e.g., Python's PyPI and Ubuntu's apt). Xie et al.[**?**] tackled this with Hera, a tool that built an offline cross-repository compatibility database. Hera addressed the cross-repository gap by modeling dependencies across PyPI and apt, and could predict, detect, and fix conflicts arising from mixed package managers—an issue that single-repository methods were unable to handle.

In domain-specific contexts, Zhao et al.[**?**] extended compatibility detection to deep learning (DL) projects, a space where general-purpose methods (e.g., DeBBI, Hera) struggled due to DL's complex stack (libraries, runtime, drivers, OS, hardware). Zhao et al. constructed a weighted knowledge graph from Stack Overflow discussions to capture DL-specific incompatibilities, but this method relied on pre-extracted, manually validated knowledge, lacking dynamic semantic reasoning. Similarly, Liu et al.[**?**] conducted a comparative study of nine Android API compatibility tools, revealing fragmentation in tool capabilities across device-specific and evolution-induced issues, but like Zhao et al.'s work, it did not leverage advanced semantic models to enhance detection. Even Zhang et al.[**?**]'s targeted work on Python variadic parameter pitfalls— which supplemented general static analysis with language-specific insights—remained confined to syntactic pattern matching.

Across all these traditional methods, a unifying limitation persisted: none leveraged Large Language Models (LLMs) for advanced semantic analysis. These approaches failed to harness LLMs' strengths in contextual reasoning and nuanced semantic understanding. In contrast, our work addresses this critical gap by integrating LLMs, enhanced via Chain-of-Thought (CoT) reasoning and multi-agent collaboration. This approach capitalizes on LLMs' ability to interpret implicit API behaviors, infer context-dependent compatibility rules, and adapt to diverse usage scenarios—capabilities that resolve the semantic detection limitations of all preceding traditional methods.

B. LLMs in the Context of API Evolution

Research on LLM-driven reasoning enhancement and practical task adaptation has yielded insights directly relevant to API evolution scenarios, with a clear trajectory of addressing prior limitations to improve adaptability and precision.

Building on the foundational work of Wei et al. (2022)[**?**], who pioneered Chain-of-Thought (CoT) prompting, LLMs gained the ability to perform multi-step reasoning—an essential capability for untangling the complex logic of API evolution (e.g., tracking parameter changes, dependency shifts). However, CoT's unstructured reasoning steps often led to ambiguity in code-related tasks, a gap addressed by Li et al. (2025)[**?**] with their Structured CoT (SCoT) prompting. By integrating programming's core structures (sequential, branch, loop) into reasoning steps, SCoT significantly boosted code generation accuracy, directly supporting code adaptation in dynamically evolving API environments where structural consistency is critical. Complementing SCoT's structural optimization, Zhang et al. (2025)[**?**] identified another limitation of CoT: its reliance on semantically similar demonstrations, which hinders robustness across diverse reasoning scenarios. To resolve this, they proposed Pattern-CoT, which extracts task-specific reasoning patterns (e.g., arithmetic operations, logical judgments) to select diverse demonstrations. This approach enhanced LLMs' ability to generalize across varied API evolution contexts, such as handling both parameter adjustments and return type modifications.

While structural and demonstration-related optimizations advanced CoT, Liu et al. (2024)[**?**] noted a lack of focus on relational understanding—key for tracking associated modifications in API evolution (e.g., how a change in one API affects dependent methods). Their ERA-CoT framework leverages entity relationship analysis to strengthen LLMs' grasp of relational changes, filling this gap and improving the detection of cascading impacts in evolving API ecosystems.

In practical scenarios tied to API evolution, such as vulnerability detection induced by API changes, Ma et al. (2024)[**?**] developed iAudit, a framework combining LoRA fine-tuning and LLM-based agents (Ranker/Critic) for smart contract auditing. Similarly, Sun et al. (2024)[**?**] proposed GPTScan, which fuses GPT's code understanding with static analysis to detect logic vulnerabilities in smart contracts—both works demonstrate the value of LLM integration with specialized analysis for evolution-related risk detection. However, neither targets API compatibility directly, focusing instead on general vulnerability auditing.

A critical limitation across these LLM-based approaches was highlighted by Stechly et al. (2024)[**?**]: CoT and its variants rely heavily on task-specific prompts, lacking adaptability in dynamic API evolution scenarios where requirements (e.g., Python's variadic parameter changes, version-specific trait shifts) are constantly evolving. More notably, none of these studies design prompts tailored to API compatibility analysis, nor do they prioritize the unique characteristics of Python APIs (e.g., dynamic imports, alias handling, variadic

parameters)—a key gap given Python's dominance in third-party library ecosystems. To address these shortcomings, our work crafts API compatibility-focused prompts that integrate Python-specific API traits (e.g., version-dependent parameter constraints, variadic parameter behaviors) and optimizes LLM reasoning for Python's dynamic features. By combining CoT's multi-step reasoning with Python API evolution insights and multi-agent collaboration, we enable precise semantic analysis of compatibility issues—filling the void of LLM-based solutions tailored to Python API compatibility detection.

REFERENCES

[1] L. Chen, F. Hassan, X. Wang, and L. Zhang, "Taming behavioral backward incompatibilities via cross-project testing and analysis," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 112–124.

[2] M. Lamothe, Y.-G. Guéhéneuc, and W. Shang, "A systematic review of api evolution literature," *ACM Computing Surveys (CSUR)*, vol. 54, no. 8, pp. 1–36, 2021.

[3] J. Li, G. Li, Y. Li, and Z. Jin, "Structured chain-of-thought prompting for code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, pp. 1–23, 2025.

[4] P. Liu, Y. Zhao, H. Cai, M. Fazzini, J. Grundy, and L. Li, "Automatically detecting api-induced compatibility issues in android apps: a comparative analysis (replicability study)," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 617–628.

[5] Y. Liu, X. Peng, T. Du, J. Yin, W. Liu, and X. Zhang, "Era-cot: improving chain-of-thought through entity relationship analysis," *arXiv preprint arXiv:2403.06932*, 2024.

[6] W. Ma, D. Wu, Y. Sun, T. Wang, S. Liu, J. Zhang, Y. Xue, and Y. Liu, "Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications," *arXiv preprint arXiv:2403.16073*, 2024.

[7] H. Quan, J. Wang, B. Li, X. Du, K. Liu, and L. Li, "Characterizing python method evolution with pymevol: An essential step towards enabling reliable software systems," in *2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2022, pp. 81–86.

[8] K. Stechly, K. Valmeekam, and S. Kambhampati, "Chain of thoughtlessness? an analysis of cot in planning," *Advances in Neural Information Processing Systems*, vol. 37, pp. 29 106–29 141, 2024.

[9] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[11] Y. Xie, Z. Jia, S. Li, Y. Wang, J. Ma, X. Li, H. Liu, Y. Fu, and X. Liao, "How to pet a two-headed snake? solving cross-repository compatibility issues with hera," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 694–705.

[12] S. Zhang, G. He, and G. Xiao, "Coding pitfalls: Demystifying the potential api compatibility risk of variadic parameters in python," in *2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2024, pp. 105–106.

[13] Y. Zhang, X. Wang, L. Wu, and J. Wang, "Enhancing chain of thought prompting in large language models via reasoning patterns," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 24, 2025, pp. 25 985–25 993.

[14] Z. Zhao, B. Kou, M. Y. Ibrahim, M. Chen, and T. Zhang, "Knowledge-based version incompatibility detection for deep learning," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 708–719.

[15] , , , and , " python api ," , vol. 36, no. 4, pp. 1435–1460, 2024.